Практическая 1 (Саттаров Булат Рамилевич ЭФМО-01-25)

Задание

Загрузите файл Гиперспектр кукурузы.csv (с помощью pd.read_csv).

1. Вычислите аналитическим путем оценки коэффициентов парной линейной регрессии и постройте ее график.
2. Найдите оценки параметров модели парной лиенйной регрессии с использованием метода градиентного спуска. Постройте график.
3. Оцените качество построенных моделей сравнив на графике обучающую выборку и прогнозы.

```
#Загрузим необходимые библиотеки
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

#Загрузим файл Гиперспектр кукурузы.csv
gipers = pd.read_csv("Гиперспектр кукурузы.csv", sep=';', decimal=',')
gipers.head()
```

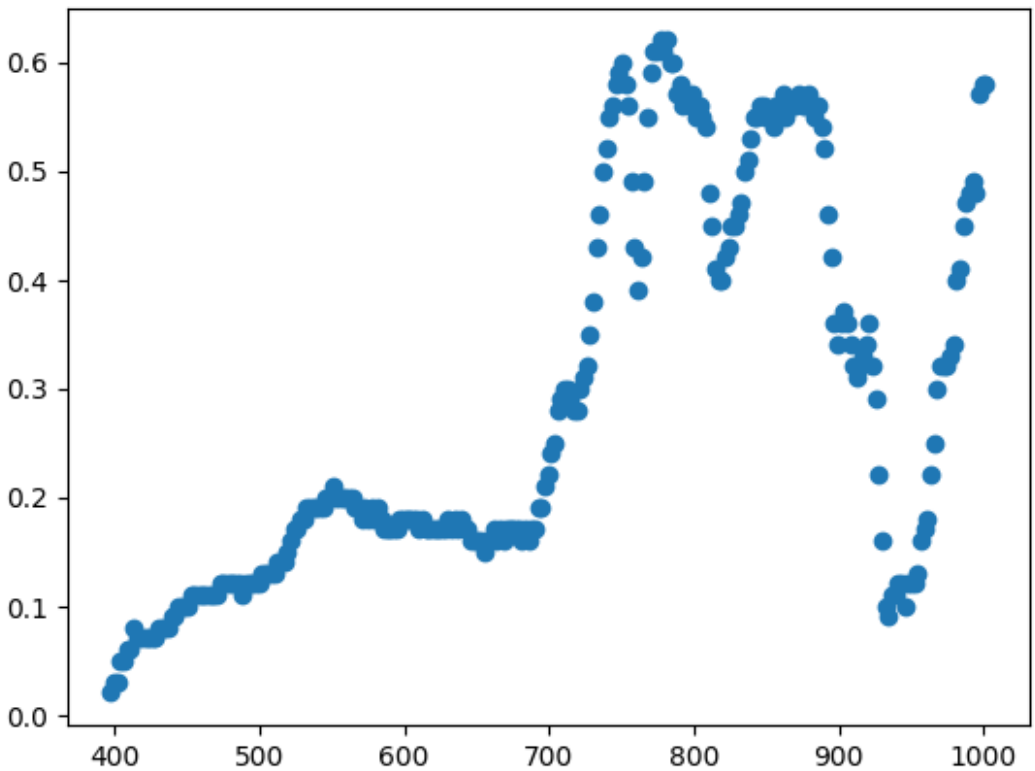
```
   wavelength  Spectr
0      397.32    0.02
1      399.54    0.03
2      401.76    0.03
3      403.98    0.05
4      406.20    0.05
```

```
#Исходные данные
x = gipers['wavelength']
Y = gipers['Spectr']
display(x, Y)
```

```
0       397.32
1       399.54
2       401.76
3       403.98
4       406.20
        ...
268     992.28
269     994.50
270     996.72
271     998.94
272    1001.16
Name: wavelength, Length: 273, dtype: float64
```

```
0      0.02
1      0.03
2      0.03
3      0.05
4      0.05
       ...
268    0.49
269    0.48
270    0.57
271    0.58
272    0.58
Name: Spectr, Length: 273, dtype: float64
```

```
#Изобразим их на графике
plt.figure()
plt.scatter(x, Y)
plt.show()
```




```
#Вычислим оценки коэффициентов парной линейной регрессии по формулам
a1 = ((x - x.mean())*(Y - Y.mean())).mean()/((x - x.mean())**2).mean()
a0 = Y.mean() - a1*x.mean()
print("Модель линейной регрессии: Y^ = ", a0, " + ", a1, "* x")
```

```
Модель линейной регрессии: Y^ =  -0.1834278267811973  +
0.0006681550262486285 * x
```

```
#Дадим серию прогнозов Y^ для x
x_space = np.linspace(x.min(), x.max(), x.size)
print(x_space)
print()
Y_pred = a0 + a1*x_space
print(Y_pred)
```

```
[ 397.32  399.54  401.76  403.98  406.2   408.42  410.64  412.86
415.08
  417.3   419.52  421.74  423.96  426.18  428.4   430.62  432.84
435.06
  437.28  439.5   441.72  443.94  446.16  448.38  450.6   452.82
455.04
  457.26  459.48  461.7   463.92  466.14  468.36  470.58  472.8
475.02
  477.24  479.46  481.68  483.9   486.12  488.34  490.56  492.78  495.
  497.22  499.44  501.66  503.88  506.1   508.32  510.54  512.76
514.98
  517.2   519.42  521.64  523.86  526.08  528.3   530.52  532.74
534.96
  537.18  539.4   541.62  543.84  546.06  548.28  550.5   552.72
554.94
  557.16  559.38  561.6   563.82  566.04  568.26  570.48  572.7
574.92
  577.14  579.36  581.58  583.8   586.02  588.24  590.46  592.68
594.9
  597.12  599.34  601.56  603.78  606.    608.22  610.44  612.66
614.88
  617.1   619.32  621.54  623.76  625.98  628.2   630.42  632.64
634.86
  637.08  639.3   641.52  643.74  645.96  648.18  650.4   652.62
654.84
  657.06  659.28  661.5   663.72  665.94  668.16  670.38  672.6
674.82
  677.04  679.26  681.48  683.7   685.92  688.14  690.36  692.58
694.8
  697.02  699.24  701.46  703.68  705.9   708.12  710.34  712.56
714.78
  717.    719.22  721.44  723.66  725.88  728.1   730.32  732.54
734.76
  736.98  739.2   741.42  743.64  745.86  748.08  750.3   752.52
754.74
  756.96  759.18  761.4   763.62  765.84  768.06  770.28  772.5
774.72
  776.94  779.16  781.38  783.6   785.82  788.04  790.26  792.48
794.7
  796.92  799.14  801.36  803.58  805.8   808.02  810.24  812.46
814.68
  816.9   819.12  821.34  823.56  825.78  828.    830.22  832.44
834.66
```

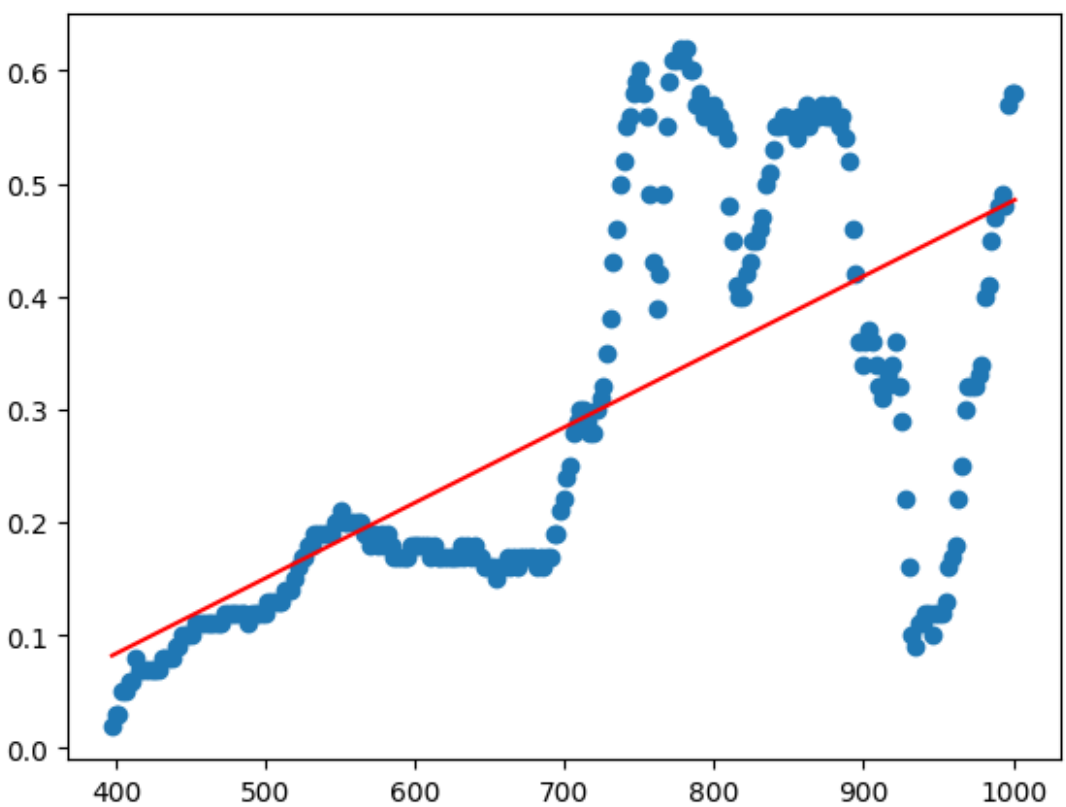
```
  836.88  839.1   841.32  843.54  845.76  847.98  850.2   852.42
854.64
  856.86  859.08  861.3   863.52  865.74  867.96  870.18  872.4
874.62
  876.84  879.06  881.28  883.5   885.72  887.94  890.16  892.38
894.6
  896.82  899.04  901.26  903.48  905.7   907.92  910.14  912.36
914.58
  916.8   919.02  921.24  923.46  925.68  927.9   930.12  932.34
934.56
  936.78  939.    941.22  943.44  945.66  947.88  950.1   952.32
954.54
  956.76  958.98  961.2   963.42  965.64  967.86  970.08  972.3
974.52
  976.74  978.96  981.18  983.4   985.62  987.84  990.06  992.28
994.5
  996.72  998.94 1001.16]

[0.08204353 0.08352683 0.08501014 0.08649344 0.08797674 0.08946005
 0.09094335 0.09242666 0.09390996 0.09539327 0.09687657 0.09835987
 0.09984318 0.10132648 0.10280979 0.10429309 0.10577639 0.1072597
 0.108743   0.11022631 0.11170961 0.11319292 0.11467622 0.11615952
 0.11764283 0.11912613 0.12060944 0.12209274 0.12357604 0.12505935
 0.12654265 0.12802596 0.12950926 0.13099257 0.13247587 0.13395917
 0.13544248 0.13692578 0.13840909 0.13989239 0.14137569 0.142859
 0.1443423  0.14582561 0.14730891 0.14879222 0.15027552 0.15175882
 0.15324213 0.15472543 0.15620874 0.15769204 0.15917534 0.16065865
 0.16214195 0.16362526 0.16510856 0.16659187 0.16807517 0.16955847
 0.17104178 0.17252508 0.17400839 0.17549169 0.17697499 0.1784583
 0.1799416  0.18142491 0.18290821 0.18439152 0.18587482 0.18735812
 0.18884143 0.19032473 0.19180804 0.19329134 0.19477464 0.19625795
 0.19774125 0.19922456 0.20070786 0.20219117 0.20367447 0.20515777
 0.20664108 0.20812438 0.20960769 0.21109099 0.21257429 0.2140576
 0.2155409  0.21702421 0.21850751 0.21999081 0.22147412 0.22295742
 0.22444073 0.22592403 0.22740734 0.22889064 0.23037394 0.23185725
 0.23334055 0.23482386 0.23630716 0.23779046 0.23927377 0.24075707
 0.24224038 0.24372368 0.24520699 0.24669029 0.24817359 0.2496569
 0.2511402  0.25262351 0.25410681 0.25559011 0.25707342 0.25855672
 0.26004003 0.26152333 0.26300664 0.26448994 0.26597324 0.26745655
 0.26893985 0.27042316 0.27190646 0.27338976 0.27487307 0.27635637
 0.27783968 0.27932298 0.28080629 0.28228959 0.28377289 0.2852562
 0.2867395  0.28822281 0.28970611 0.29118941 0.29267272 0.29415602
 0.29563933 0.29712263 0.29860594 0.30008924 0.30157254 0.30305585
 0.30453915 0.30602246 0.30750576 0.30898906 0.31047237 0.31195567
 0.31343898 0.31492228 0.31640559 0.31788889 0.31937219 0.3208555
 0.3223388  0.32382211 0.32530541 0.32678871 0.32827202 0.32975532
 0.33123863 0.33272193 0.33420524 0.33568854 0.33717184 0.33865515
 0.34013845 0.34162176 0.34310506 0.34458836 0.34607167 0.34755497
 0.34903828 0.35052158 0.35200489 0.35348819 0.35497149 0.3564548
```

```
0.3579381  0.35942141 0.36090471 0.36238801 0.36387132 0.36535462
0.36683793 0.36832123 0.36980453 0.37128784 0.37277114 0.37425445
0.37573775 0.37722106 0.37870436 0.38018766 0.38167097 0.38315427
0.38463758 0.38612088 0.38760418 0.38908749 0.39057079 0.3920541
0.3935374  0.39502071 0.39650401 0.39798731 0.39947062 0.40095392
0.40243723 0.40392053 0.40540383 0.40688714 0.40837044 0.40985375
0.41133705 0.41282036 0.41430366 0.41578696 0.41727027 0.41875357
0.42023688 0.42172018 0.42320348 0.42468679 0.42617009 0.4276534
0.4291367  0.43062001 0.43210331 0.43358661 0.43506992 0.43655322
0.43803653 0.43951983 0.44100313 0.44248644 0.44396974 0.44545305
0.44693635 0.44841966 0.44990296 0.45138626 0.45286957 0.45435287
0.45583618 0.45731948 0.45880278 0.46028609 0.46176939 0.4632527
0.464736   0.46621931 0.46770261 0.46918591 0.47066922 0.47215252
0.47363583 0.47511913 0.47660243 0.47808574 0.47956904 0.48105235
0.48253565 0.48401896 0.48550226]
```

```
#Изобразим на графике исходные данные и прогнозы
fig  = plt.figure()
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])
ax.scatter(x, Y)
ax.plot(x_space, Y_pred, 'r')
```

```
[<matplotlib.lines.Line2D at 0x10c2122b0>]
```



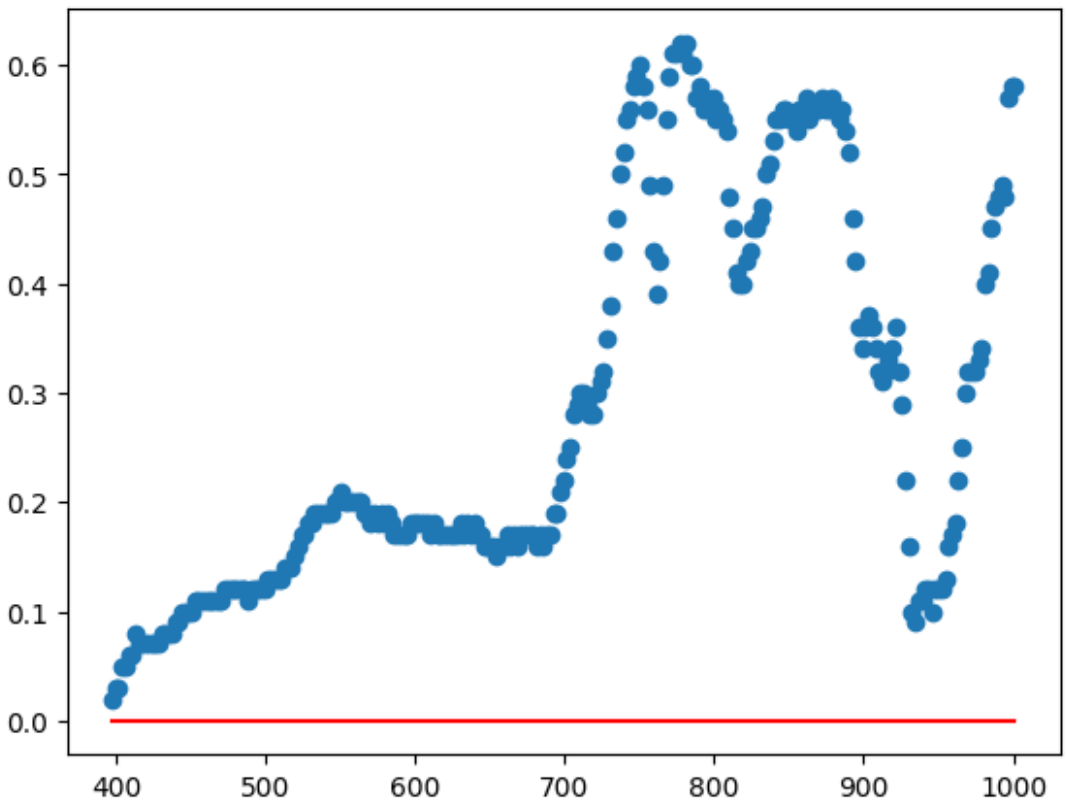
```
#Реализуем шаг градиентного спуска в модели парной линейной регрессии
class SimpleRegression(object):
    def __init__(self):
        self.a0 = 0
        self.a1 = 0
    def predict(self, x):
        return self.a0 + self.a1*x
    def MSE(self, x, Y):
        return ((self.predict(x)-Y)**2).mean()
    def fit(self, x, Y):
        alpha = 1e-12
        dT_a0 = -2*sum((Y - self.predict(x)))
        dT_a1 = -2*sum((Y - self.predict(x))*x)
        self.a0 -= alpha*dT_a0
        self.a1 -= alpha*dT_a1

#Получим прогнозы до градиентного спуска с начальными значениями
параметров
regr = SimpleRegression()
print(regr.predict(x.iloc[0]))
print(regr.predict(x.iloc[1]))
print(regr.predict(x.iloc[2]))
print(regr.predict(x.iloc[3]))
print(regr.predict(x.iloc[4]))
print(regr.MSE(x, Y))
```

```
0.0
0.0
0.0
0.0
0.0
0.11187582417582417
```

```
#Выведем прогнозы до градиентного спуска на графике
x_space = np.linspace(x.min(), x.max(), 100)
Y_pred = regr.predict(x_space)
fig = plt.figure()
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])
ax.scatter(x, Y)
ax.plot(x_space, Y_pred, 'r')
```

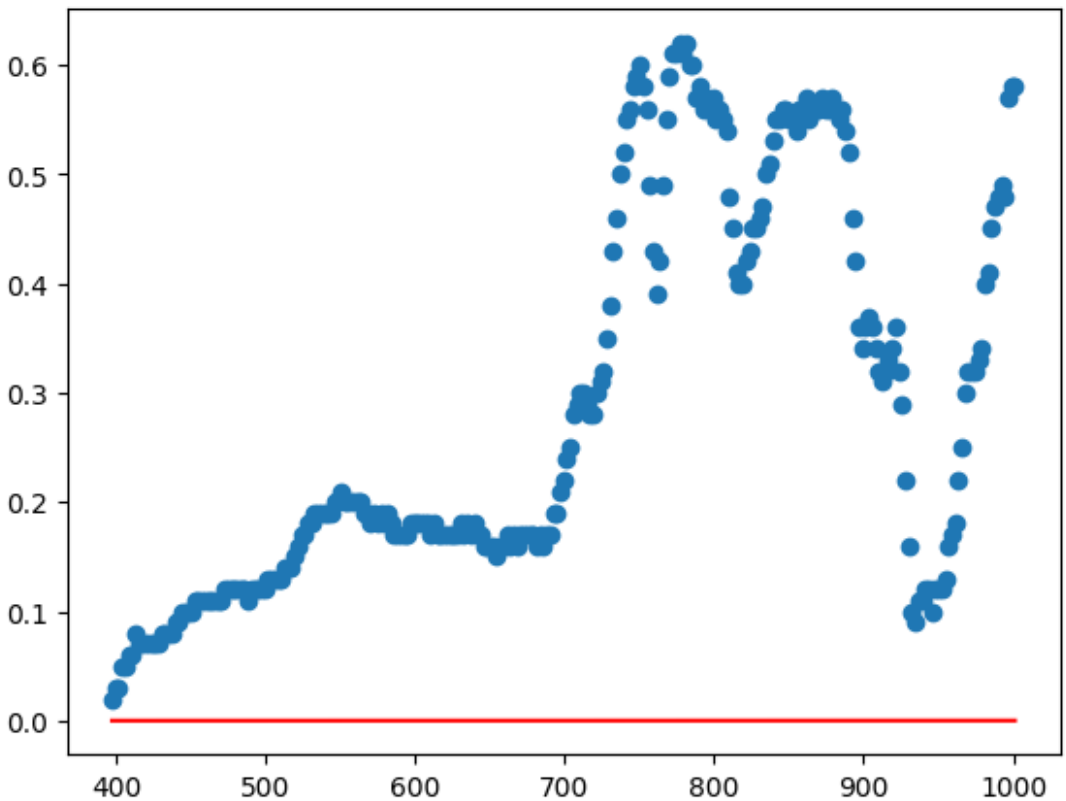
```
[<matplotlib.lines.Line2D at 0x10cc5da00>]
```




```
#Реализуем шаг градиентного спуска
regr.fit(x, Y)
print("MSE после первого шага градиентного спуска: ", regr.MSE(x, Y))
Y_pred = regr.predict(x_space)
fig = plt.figure()
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])
ax.scatter(x, Y)
ax.plot(x_space, Y_pred, 'r')
```

```
MSE после первого шага градиентного спуска:  0.11182351706220367
```

```
[<matplotlib.lines.Line2D at 0x10c9dc0d0>]
```




```
#Реализуем цикл градиентного спуска
class SimpleRegression(object):
    def __init__(self):
        self.a0 = 0
        self.a1 = 0
    def predict(self, x):
        return self.a0 + self.a1*x
    def MSE(self, x, Y):
        return ((Y - self.predict(x))**2).mean()
    def MAE(self, x, Y):
        return abs(Y - self.predict(x)).mean()
    def MAPE(self, x, Y):
        return abs((Y - self.predict(x))/Y).mean()
    def fit(self, x, Y, alpha = 0.00000001, epsylon = 0.001, max_steps
= 200000):
        steps, errors = [], []
        step = 0
        for _ in range(max_steps):
            dT_a0 = -2*sum((Y - self.predict(x)))
            dT_a1 = -2*sum((Y - self.predict(x))*x)
            self.a0 -= alpha*dT_a0
            self.a1 -= alpha*dT_a1
            new_error = self.MSE(x, Y)
            step += 1
```

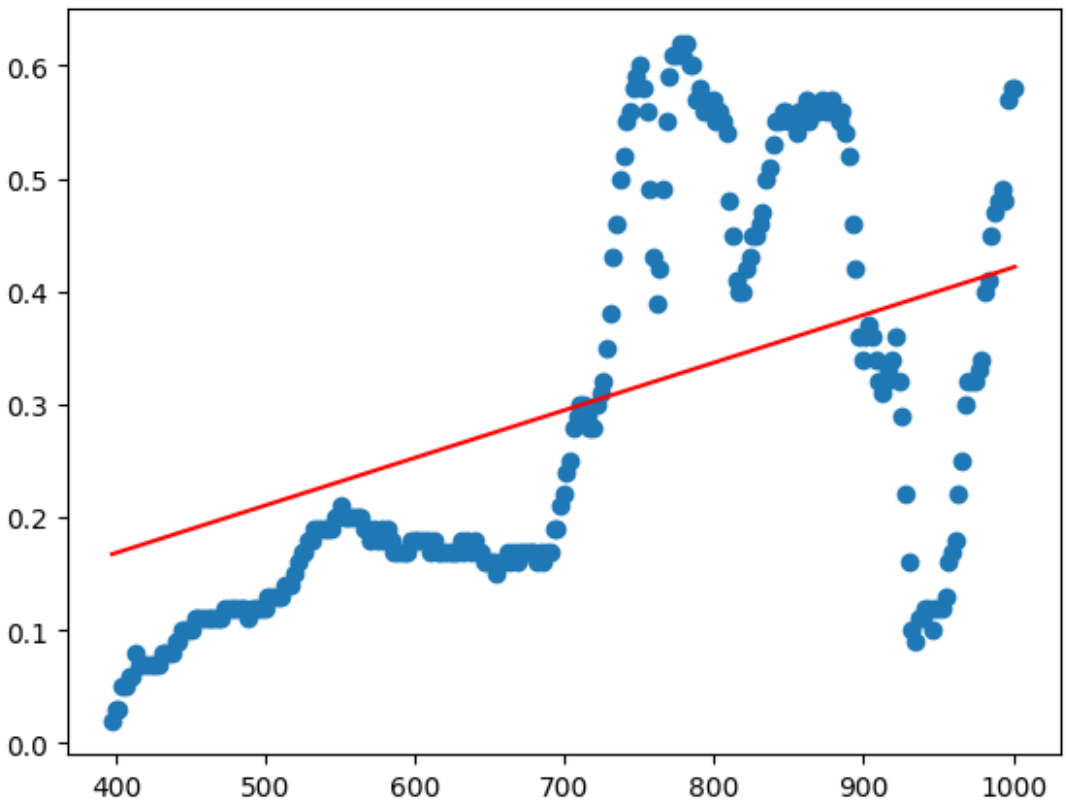
```
            steps.append(step)
            errors.append(new_error)
            if new_error < epsylon:
                break
        return steps, errors

#Запустим цикл градиентного спуска с заданной точностью 0.001
regr = SimpleRegression()
steps, errors = regr.fit(x, Y, alpha = 1e-12, epsylon = 0.001,
max_steps = 200000)

#Выведем график прогнозов и вычислим MSE
print(regr.predict(x.iloc[0]))
print(regr.predict(x.iloc[1]))
print(regr.predict(x.iloc[2]))
print(regr.predict(x.iloc[3]))
print(regr.predict(x.iloc[4]))
print("MSE после градиентного спуска: ", regr.MSE(x, Y))
Y_pred = regr.predict(x_space)
fig = plt.figure()
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])
ax.scatter(x, Y)
ax.plot(x_space, Y_pred, 'r')
```

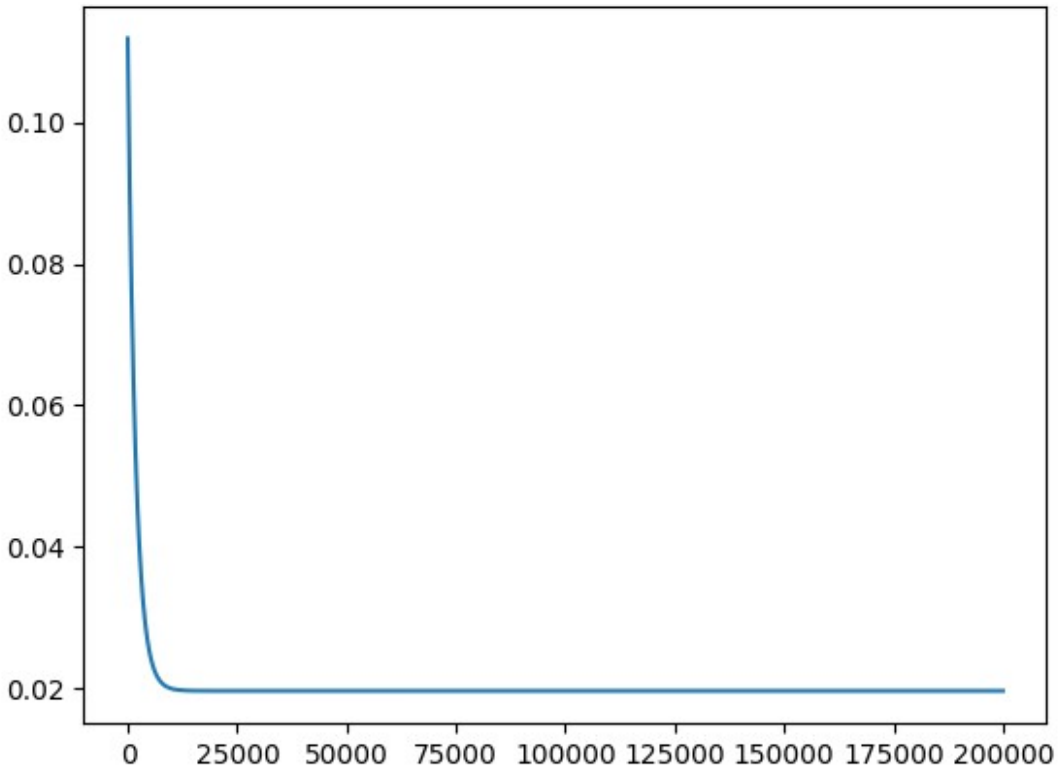
```
0.16738471358259097
0.16831996835250684
0.1692552231224227
0.17019047789233857
0.17112573266225442
MSE после градиентного спуска:  0.019666318629093335
```

```
[<matplotlib.lines.Line2D at 0x10db61be0>]
```




```
#Выведем график изменения MSE в процессе градиентного спуска
plt.figure()
plt.plot(steps, errors)
```

```
[<matplotlib.lines.Line2D at 0x10db354f0>]
```



Сравнение моделей

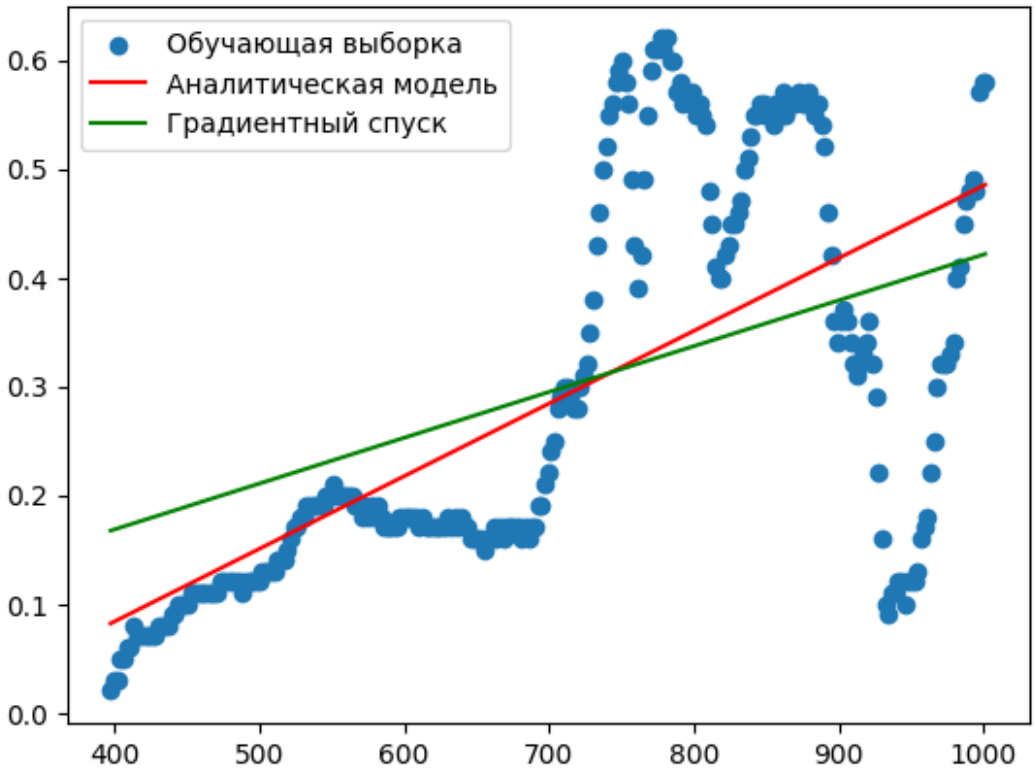
```
Y_pred_analytic = a0 + a1*x
Y_pred_grad = regr.predict(x)

MSE_analytic = ((Y - Y_pred_analytic)**2).mean()
MSE_grad = ((Y - Y_pred_grad)**2).mean()

print("MSE аналитической модели: ", MSE_analytic)
print("MSE модели после градиентного спуска: ", MSE_grad)

plt.figure()
plt.scatter(x, Y, label='Обучающая выборка')
plt.plot(x, Y_pred_analytic, 'r', label='Аналитическая модель')
plt.plot(x, Y_pred_grad, 'g', label='Градиентный спуск')
plt.legend()
plt.show()
```

```
MSE аналитической модели:  0.017684114628301398
MSE модели после градиентного спуска:  0.019666318629093335
```


Обучающая выборка
Аналитическая модель
Градиентный спуск
0.0
0.1
0.2
0.3
0.4
0.5
0.6
400
500
600
700
800
900
1000